Advanced Git Workflows & Strategies

Workflows • Rollback Strategies • Tags • Bisect • Real-World Scenarios

Review

Why Branches & The 3 Types

Why branch?

- Work on features in parallel without breaking the stable codebase.
- Isolate experiments — if it fails, just delete the branch.
- Enable code review via Pull Requests before merging.

Local Non-Tracking

`git branch my-feature`

Exists only locally. Not linked to any remote.

Local Tracking

`git push -u origin my-feature`

Local branch linked to a remote counterpart. Enables simple git pull / git push.

Remote Tracking

`git fetch → origin/main`

Read-only reference mirroring the remote state. Updated by git fetch.

Review

Push / Pull Branches to GitHub

git push

`$ git push origin main`
Upload local commits to remote

`$ git push -u origin feature`
Push + set upstream tracking

`$ git push`
Works once upstream is set (-u)

git pull

`$ git pull origin main`
Download + merge remote changes

`$ git pull`
Same as: git fetch + git merge

`$ git fetch origin`
Download only (no merge) — safer

💡 *git pull = git fetch + git merge. When in doubt, fetch first to see what changed, then merge manually.*

Review

Merge Conflicts

A conflict occurs when two branches modify the same line(s) in a file. Git can't auto-merge — you decide.

```
<<<<<<< HEAD (your changes)
banana
=======
mango
>>>>>>> main (incoming changes)
```

How to resolve:

1. Open the conflicted file — VS Code highlights conflicts with colored blocks.
2. Choose: Accept Current, Accept Incoming, Accept Both, or edit manually.
3. Remove all conflict markers (<<<, ===, >>>). Save the file.
4. git add <file> → git commit → done!

Today's Agenda

Git Workflows
Feature Branch Flow, Gitflow, Trunk-Based Development

Branching Strategies
When to branch, naming conventions, real-world patterns

Rollback Strategies
reset, revert, cherry-pick, rebase -i — when to use which

Git Tags
Lightweight vs annotated, semantic versioning, release workflow

Git Bisect & Other Tools
Binary search for bugs, stash, useful commands

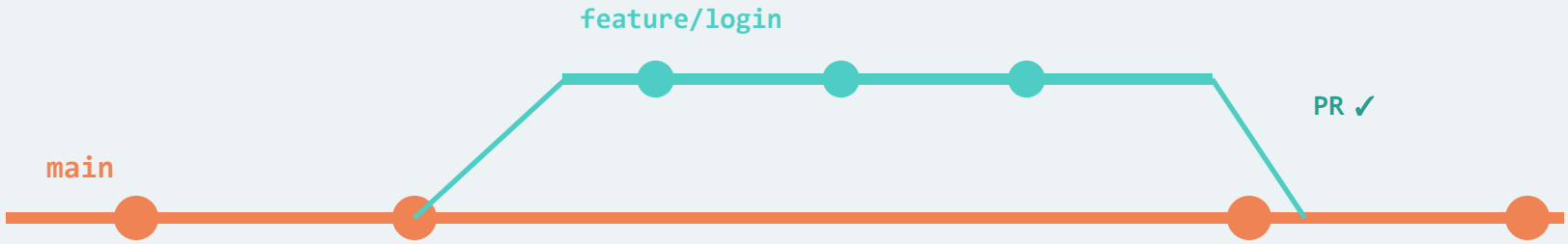
Real-World Scenarios
Hotfixes, broken deploys, messy histories, exercises

Git Workflows

How teams organize branches and collaborate

Feature Branch Flow

The simplest workflow. One branch per feature, merged into main via Pull Request.



How it works

Create a branch from main → work on feature → open PR → code review → merge.

Best for

Small teams, simple projects, continuous deployment setups.

Branch naming

feature/*, bugfix/*, hotfix/* — descriptive and prefixed.

Key rule

main is always deployable. Never commit directly to main.

Gitflow

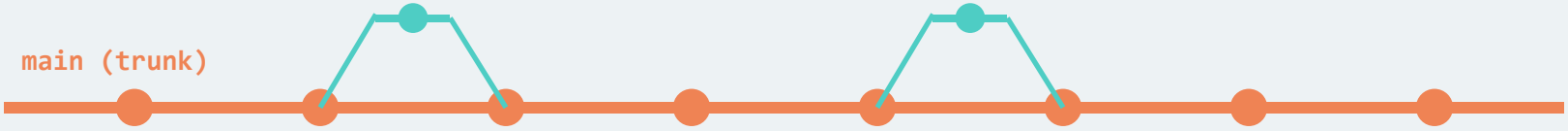
A structured workflow with dedicated branches for different stages of development.

`main`	Production-ready code. Only receives merges from release and hotfix.
`develop`	Integration branch. All features merge here first.
`feature/*`	Branch from develop. One per feature. Merge back into develop.
`release/*`	Branch from develop when ready. Bug fixes only. Merge into main + develop.
`hotfix/*`	Branch from main for urgent fixes. Merge into main + develop.

💡 *Best for teams with scheduled releases. Can be overkill for small projects or continuous delivery.*

Trunk-Based Development

Everyone commits to main (the "trunk") frequently. Short-lived branches only.



Short-lived branches

Branches live hours to 1-2 days max, not weeks.

Continuous integration

Merge often, test always. CI pipeline is critical.

Feature flags

Ship incomplete features behind flags, toggle in production.

Best for

Mature teams with strong CI/CD, fast iteration cycles.

Workflow Comparison

	Feature Branch	Gitflow	Trunk-Based
Complexity	Low	High	Medium
Branch lifetime	Days–weeks	Days–weeks	Hours–1 day
Release model	Continuous	Scheduled	Continuous
Team size	Any	Medium–large	Any (mature)
CI/CD required	Recommended	Recommended	Essential
Merge conflicts	Moderate	Moderate–High	Low
Rollback ease	Moderate	Good (revert release)	Very Good

💡 *There's no single "best" workflow — choose based on team maturity, release cadence, and project needs.*

Rollback Strategies

When things go wrong — and they will — how do you undo?

Which Rollback Tool?

Pick the right tool based on the situation:

Undo last commit, keep changes staged	→ `git reset --soft HEAD~1`
Undo last commit, discard everything	→ `git reset --hard HEAD~1`
Undo a commit already pushed to shared branch	→ `git revert <hash>`
Grab one specific commit from another branch	→ `git cherry-pick <hash>`
Clean up / squash / reorder local commits	→ `git rebase -i HEAD~N`

git reset — soft vs. hard

--soft

```
$ git reset --soft HEAD~1
```

Moves HEAD back but keeps all changes staged.

Use case: Undo a commit to split it into smaller ones, fix the message, or add more changes.

--hard

```
$ git reset --hard HEAD~1
```

Moves HEAD back and discards all changes completely.

Use case: Completely throw away commits as if they never happened.

⚠️ **Destructive — changes are lost!**

💡 *Rule of thumb: use --soft to redo a commit, use --hard only on local unpushed work you're sure you want gone.*

git revert — The Safe Undo

Creates a NEW commit that undoes the changes from a specific commit. History is preserved.

```
# Revert a single commit
$ git revert abc1234

# Revert without auto-commit (stage changes only)
$ git revert --no-commit abc1234

# Revert a merge commit (specify parent)
$ git revert -m 1 <merge-commit-hash>
```

✓ When to use

Commit is already pushed. Other people have the history.

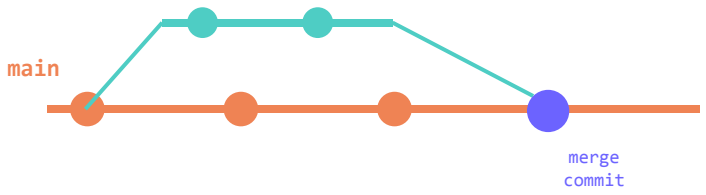
✗ When NOT to use

You want a clean history with no trace. Use reset instead (local only).

git rebase — Update Your Branch

Replay your feature branch commits on top of the latest main. Cleaner alternative to merge.

git merge main



Creates a merge commit.
History shows the branch existed.

git rebase main

No merge commit.
Clean, linear history.

```
$ git checkout feature/login        # switch to your branch
$ git rebase main                    # replay your commits on top of main
```

git rebase -i — Interactive Rebase

Rewrite local history: squash, reorder, reword, or drop commits before pushing.

```
$ git rebase -i HEAD~4

# Editor opens — change the action word for each commit:
pick   abc1234 Add user model              ← keep as-is
squash def5678 Fix typo in user model      ← merge into previous commit
reword ghi9012 Add validation              ← keep but edit the message
drop   jkl3456 Debug logging               ← remove entirely
```

Available actions:

`pick (p)`	Keep the commit as-is
`reword (r)`	Keep but change the commit message
`squash (s)`	Merge into the previous commit (combine messages)
`fixup (f)`	Like squash but discard this commit's message
`drop (d)`	Remove the commit entirely

git cherry-pick

Copy specific commits from one branch onto another — without merging the whole branch.

```
# Copy a single commit
$ git cherry-pick abc1234

# Copy multiple commits
$ git cherry-pick abc1234 def5678 ghi9012

# Cherry-pick without auto-committing (stage only)
$ git cherry-pick --no-commit abc1234
```

Hotfix from feature branch	A bugfix is on a feature branch but you need it on main now — cherry-pick just that commit.
Recover lost commits	Commits were removed by a reset or force-push — cherry-pick them back from reflog.
Backport to older version	A fix on main needs to go to a release/v1.x branch — cherry-pick it there.

Exercise: Rollback Practice

1. Create a new repo. Add a file, commit it.
2. Make a second commit (add more content).
3. Run git reset --soft HEAD~1 — verify the changes are staged (git status).
4. Recommit with a different message.
5. Run git reset --hard HEAD~1 — verify the changes are completely gone.
6. Make two new commits. Use git revert HEAD to undo the last one. Check git log.

💡 *After each step, use git log --oneline and git status to see what happened.*

Git Tags

Marking releases and important milestones

Lightweight vs. Annotated Tags

Lightweight

```
$ git tag v1.0.0
```

Just a pointer to a commit. No metadata, no message. Like a bookmark.

Use for: temporary or personal markers.

Annotated ★

```
$ git tag -a v1.0.0 -m "Release 1.0"
```

Full Git object with author, date, message, and optional GPG signature.

Use for: releases. Always prefer this.

```
$ git tag                    # list all tags
$ git push origin v1.0.0     # push a specific tag
$ git push origin --tags     # push all tags
$ git tag -d v1.0.0          # delete local tag
```

Semantic Versioning & Release Tags

v1 . 4 . 2

MAJOR	MINOR	PATCH
Breaking changes	New features (backwards-compatible)	Bug fixes

Typical release workflow:

```
$ git checkout main
$ git pull origin main
$ git tag -a v1.4.2 -m "Fix: payment timeout bug"
$ git push origin v1.4.2
```

git bisect & Other Tools

Finding bugs with binary search, stashing work, and other essentials

git bisect — Find the Bug

Binary search through commits to find exactly which one introduced a bug.

```
$ git bisect start
$ git bisect bad                    # current commit is broken
$ git bisect good v1.3.0            # this tag was working

# Git checks out a commit in the middle. You test it:
$ git bisect good                   # if this commit works
$ git bisect bad                    # if this commit is broken

# Repeat until Git finds the exact commit. Then:
$ git bisect reset                  # go back to your branch
```

💡 *With 1000 commits, bisect finds the culprit in ~10 steps ($\log_2 1000 \approx 10$). Much faster than checking one by one.*

📦 git stash — Shelve Your Work

Temporarily save uncommitted changes and restore a clean working tree.

```
$ git stash
          # save changes, clean working tree
$ git stash list
          # see all stashed entries
$ git stash apply
          # re-apply last stash (keeps it in list)
$ git stash pop
          # re-apply last stash (removes from list)
$ git stash drop stash@{0}
          # delete a specific stash entry

# apply vs pop:  apply = reusable (stash stays)    pop = one-time (stash deleted)
```

💡 *Typical use: you're mid-feature and need to switch branches for a hotfix. Stash → switch → fix → switch back → pop.*

Useful Commands Reference

`git diff`	Show unstaged changes
`git diff --staged`	Show staged changes
`git diff --stat`	Summary: files changed, lines +/-
`git checkout -- <file>` / `git restore <file>`	Discard unstaged changes (restore: Git 2.23+)
`git reset HEAD <file>` / `git restore --staged <file>`	Unstage a file (restore: Git 2.23+)
`git rm --cached <file>`	Remove from tracking without deleting the file
`git commit --amend`	Modify the last commit (message or contents)
`git log --oneline --graph --all`	Visual history of all branches
`git checkout <commit-hash>`	Inspect a specific commit (detached HEAD)

Good Commits & Good PRs

Commit Messages

```
✓ fix: resolve login timeout on slow
networks
✓ feat: add email validation to signup
✗ fixed stuff
✗ wip / asdfasdf
```

Rules:

- Imperative mood: "add" not "added"
- Subject line under 50 chars
- Explain WHY in body, not just WHAT
- Prefixes: feat, fix, refactor, docs, test, chore

Pull Requests

- Keep PRs small and focused
- One feature or one fix per PR
- Clear title and description
- Link the related issue/ticket
- Screenshots for UI changes
- Self-review before requesting others
- Squash messy commits first
- Respond to review comments promptly

🎯 Scenario Challenge

Real-world Git problems. You discuss the solution — then we reveal it.

How it works:

1. Read the scenario
2. Discuss with your neighbor — what commands would you use?
3. We reveal and walk through the solution together

🎯 CHALLENGE 1

Production Is Down! 🔥

The situation:

Users are reporting that the checkout page crashes. The bug is live in production.
You're halfway through building a new dashboard feature on feature/dashboard.

You need to: save your current work, fix the bug on main, deploy, and go back to your feature.

What do you do?

✓ SOLUTION 1

Production Hotfix

```
# 1. Save your current work
$ git stash

# 2. Create hotfix branch from main
$ git checkout main && git pull
$ git checkout -b hotfix/payment-crash

# 3. Fix the bug, commit, push, create PR
$ git commit -m "fix: resolve payment crash on checkout"
$ git push -u origin hotfix/payment-crash

# 4. After PR merge → tag the release → go back to your feature
$ git tag -a v1.4.3 -m "Hotfix: payment crash"
$ git push origin v1.4.3
$ git checkout feature/dashboard && git stash pop
```

💡 *In Gitflow, also merge the hotfix into develop so the fix doesn't get lost.*

🎯 CHALLENGE 2

Oops — Pushed a Broken Commit ⚠️

The situation:

You accidentally pushed a commit with a bug directly to main.
Three teammates have already pulled it.

Your gut says git reset --hard and git push --force. Is that the right call?

What do you do?

✓ SOLUTION 2

Bad Commit on Shared Branch

✗ DON'T

```
$ git reset --hard HEAD~1
$ git push --force
```

Force-pushing rewrites shared history. Others who pulled will get conflicts or lose work.

✓ DO

```
$ git checkout main && git pull
$ git checkout -b revert-buggy-commit
$ git revert <bad-commit-hash> | HEAD
$ git push
$ (PR or git merge)
```

Revert creates a new undo commit. History preserved, no one loses work.

Exception: git push --force-with-lease

Safer than --force. Only works if no one else pushed since you last fetched. Use on YOUR feature branches only, never on main.

Key takeaway: once a commit is on a shared branch, use revert — never rewrite history.

🎯 CHALLENGE 3

12 Messy Commits Before PR 🧹

The situation:

Your feature branch has 12 commits: "wip", "fix typo", "oops", "WIP again", "actually fix it".
You want a clean PR with a professional commit history.

The feature works fine — it's just the history that's embarrassing. How do you clean it up?

What do you do?

✓ SOLUTION 3

Cleaning Up Before PR

```
# Option A: Interactive rebase (surgical — pick what to keep/squash/drop)
$ git rebase -i HEAD~12
# squash fix commits, reword messages, drop debug commits

# Option B: Soft reset (nuclear — collapse everything into one commit)
$ git reset --soft main
$ git commit -m "feat: add user dashboard with charts"

# Then force-push YOUR branch (safe — it's your branch, not shared)
$ git push --force-with-lease
```

💡 *Clean commits make PRs easier to review, git blame more useful, and reverts more surgical.*

🎯 CHALLENGE 4

Committed to the Wrong Branch 😱

The situation:

You just made 2 commits with great work... but you forgot to switch (local) branches.
You're on main instead of feature/search.

The commits are local only (not pushed yet). You need them on feature/search, not main.

What do you do?

✓ SOLUTION 4

Wrong Branch Fix

Option A: Branch + Reset

```
# 1. Soft reset — moves HEAD back but keeps
your changes staged
$ git reset --soft HEAD~2
# 2. Stash the staged changes
$ git stash
# 3. Switch to the feature branch
$ git checkout feature/search
# 4. Pop the stash and commit
$ git stash pop
$ git commit -m "your message"
```

Safest approach since nothing gets discarded at any point

Option B: Cherry-pick

```
# Note the commit hashes
$ git log --oneline -3

# Switch to feature branch
$ git checkout feature/search
$ git cherry-pick abc123 def456

# Remove from main
$ git checkout main
$ git reset --hard HEAD~2
```

Use when the feature branch already exists with other work.

💡 *Both work because the commits are local only. If you'd already pushed, you'd need revert instead.*

Hands-On Exercise

Full workflow: branch, conflict, resolve, PR

Exercise Steps

1. Create repo session_example (local + GitHub). On main, create fruits.txt with 3 fruits. Push.
2. From main, create branch feature/fruits. Change the 2nd fruit, add 2 new fruits. Commit.
3. On GitHub (main branch), change the 2nd fruit's name in fruits.txt.
4. Locally, update feature/fruits with main's changes. Resolve the merge conflict.
5. Push feature/fruits to GitHub. Create a Pull Request into main.
6. After merging, tag the release: git tag -a v1.0.0 -m "First release"

★ *Bonus: use git log --oneline --graph --all to visualize the history.*

★ *Bonus: use git bisect to find which commit changed the 2nd fruit name.*